Linguaggi di Programmazione

Generare scanner con Lex.
Proprietà algoritmiche dei linguaggi regolari.

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Lex: Generatore di Scanner

LEX: Generatore di Scanner

Lex (1975, M.Lesk e E.Schmidt) è il capostipite dei generatori di analizzatori lessicali (o scanner), che gira sotto UNIX, ed è spesso usato insieme al parser generator YACC.
(**Flex** (1987, V. Paxson, free software, `github.com/westes/flex`) è una versione più recente che è spesso usato insieme al parser generator GNU Bison.)

- input: un file di tipo `.l` che contiene un insieme di pattern ed una serie di azioni corrispondenti.
- output: un programma C che realizza l'automa riconoscitore e che associa ad ogni istanza di un pattern la relativa azione.



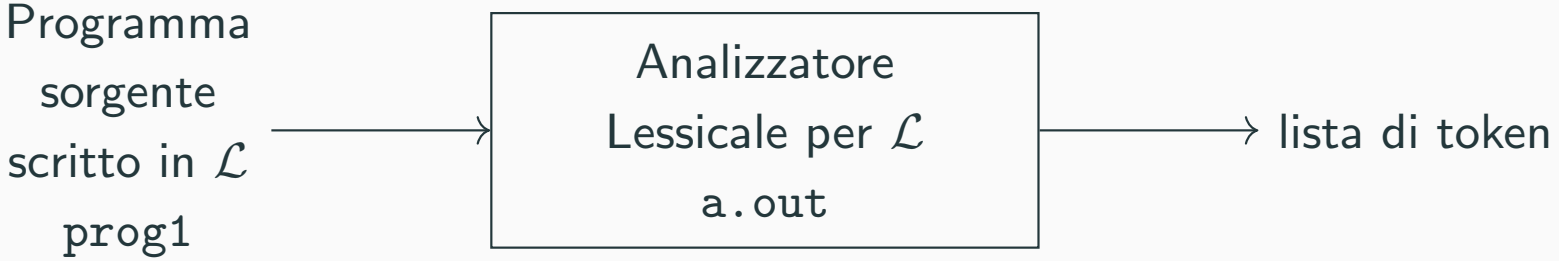

```
> lex lang.l
```

LEX: Generatore di Scanner (2)




```
> cc lex.yy.c -ll
```

per linkare le librerie di subroutine di Lex




```
> a.out prog1
```

Struttura di un file di tipo `.l`

Diviso in 3 parti:

- **Dichiarazioni**: ovvero definizioni regolari.
  - Ad es.:

    `cifra` $[0-9]$

    `cifre` $[0-9]^+$

    `ide` $[a-zA-Z]([a-zA-Z]|cifra)^*$
- **Regole**: ovvero {espressione regolare} {*azione*}

  $\underbrace{\qquad\qquad}_{pattern}$ $\underbrace{\quad}_{frammento\ di\ codice\ C}$
  - Quando viene riconosciuto un lessema di un dato pattern, verrà eseguito l'associato codice C.
  - Ad es.:

    `{cifre}` `{printf("<NUM,%s>", yytext);}`

    `{ide}` `{printf("<IDE,%s>", yytext);}`

    `yytext` è la variabile che punta al primo carattere del testo matched, la cui lunghezza è specificata da `yyleng`
- **Funzioni ausiliarie**
  - Se si usano funzioni complesse nella parte *azione*, queste potrebbero essere definite qui (codice C che viene copiato verbatim nel file `lex.yy.c`)

Come funziona un programma `lex.yy.c`?

Il programma C ottenuto `lex.yy.c` (e poi compilato per renderlo eseguibile come `a.out`) implementa essenzialmente il DFA riconoscitore dell'insieme delle espressioni regolari contenute nelle "Regole":

- scandisce il testo sorgente alla ricerca di una stringa che corrisponda a (ovvero, sia un lessema per) una delle espressioni regolari (ovvero, per un pattern di categoria sintattica);
- quando riconosce un lessema, esegue l'azione specificata (ovvero il corrispondente frammento di codice C), e passa in output il risultato dell'azione al posto del lessema;
- quando l'input non corrisponde a nessun pattern, lo lascia inalterato e "segnala" la cosa al "gestore degli errori".

Esempio banale

```
                  p1   {a}      {printf("Z");}
%% Regole         p2   {abb}    {printf("K");}
                  p3   {a*b+}   {printf("Y");}
```

Osserva che:

- il pattern p_1 è un prefisso del pattern p_2: allora il programma cerca il pattern più lungo!
- il pattern p_2 è un caso speciale del pattern p_3: allora il programma riconosce il primo in elenco!

Se l'input è `babba`, il programma `lex.yy.c` riconosce i seguenti lessemi:

```
   b    abb    a
```

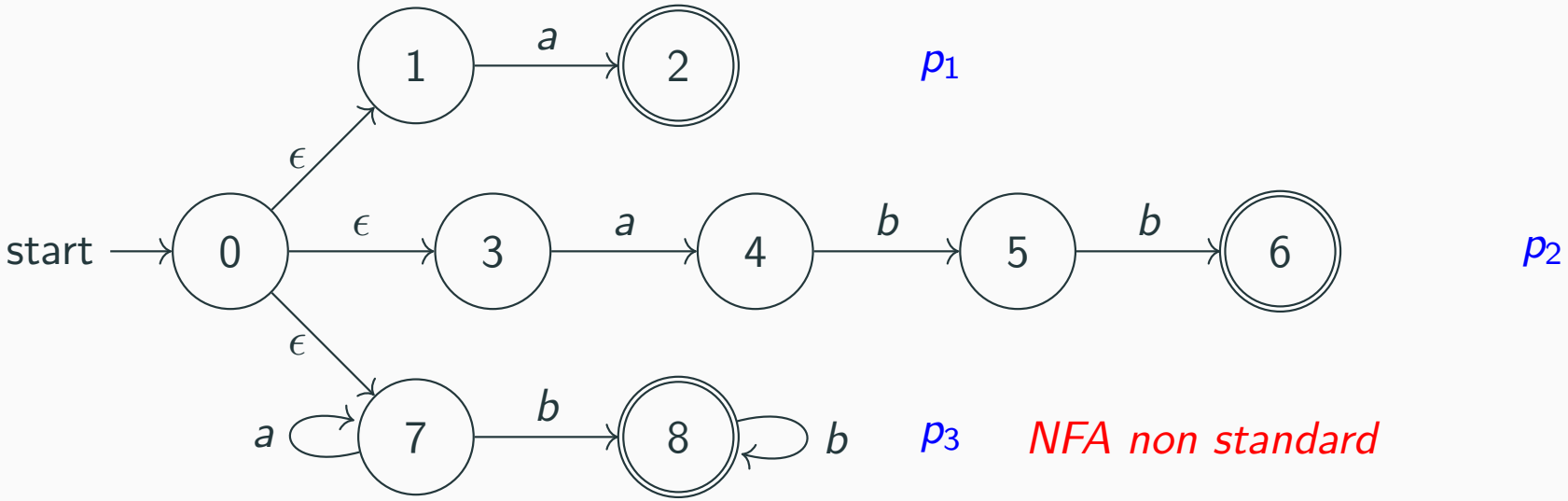
$\underbrace{\texttt{b}}_{p_3}$ $\underbrace{\texttt{abb}}_{p_2}$ $\underbrace{\texttt{a}}_{p_1}$

che "traduce" in `Y K Z`.

N.B.: Non spezza `abb` in `a` (p_1) e `bb` (p_3), perché cerca il match più lungo!

Esempio banale –continua

Ma come fa a riconoscere se un testo in input è un lessema per un pattern? Il programma `lex.yy.c` simula il funzionamento di un NFA complessivo per le tre espressioni regolari.



Fa una simulazione DFA di questo NFA, cioè parte dallo stato iniziale ϵ-*closure*$(0) = \{0, 1, 3, 7\}$. Se legge a, finisce su $\{2, 4, 7\}$, che è uno stato che riconosce il patter p_1 (a causa di 2). Se legge b, finisce su $\{8\}$, che riconosce il patter p_3.

Esempio banale – continua (2)

Stato	Simbolo input		Pattern riconosciuto	Azione corrispondente
	a	b		
0137	247	8	nessuno	–
$\underline{2}47$	7	58	$p_1 = a$	`printf("Z")`
$\underline{8}$	–	8	$p_3 = a^* b^+$	`printf("Y")`
7	7	8	nessuno	–
$5\underline{8}$	–	68	$p_3 = a^* b^+$	`printf("Y")`
$\underline{68}$	–	8	$p_2 = abb$	`printf("K")`

Il programma `lex.yy.c` procede in questo modo:

- Va avanti a leggere l'input finché non si blocca.
- Se lo stato in cui si trova è di riconoscimento, ok. Altrimenti, torna indietro fino a trovare uno stato di riconoscimento. Questo garantisce che se più pattern possono essere riconosciuti dallo stesso prefisso, il più lungo viene scelto.
- Se, arrivato al "match", vede che più di un pattern andrebbe bene, allora prende quello elencato per primo (stato 68 riconosce p_2).

Esempio banale – continua (3)

Sull'input `b a b b a`, il programma `lex.yy.c`:

- Inizia in 0137 $\xrightarrow{b}$ 8 $\not\xrightarrow{a}$: si è bloccato in 8, quindi b è un lessema del pattern $p_3 = a^* b^+$.
- L'input residuo è `a b b a`
  Ricomincia da 0137: 0137 $\xrightarrow{a}$ 247 $\xrightarrow{b}$ 58 $\xrightarrow{b}$ 68 $\not\xrightarrow{a}$: si è bloccato in 68, quindi abb è un lessema del pattern $p_2 = abb$
  - Notare che è passato su stati di riconoscimento, ma, proseguendo fino al blocco, ha preso il lessema più lungo.
  - Notare che 68 è di riconoscimento per p_2, perché viene prima in elenco di p_3, che pure faceva match.
- L'input residuo è `a`
  Ricomincia da 0137: 0137 $\xrightarrow{a}$ 247, quindi a è un lessema del pattern $p_1 = a$.

Osservazione

Nel file di tipo `.l` per un qualunque linguaggio di programmazione, le parole riservate vanno sempre messe prima degli identificatori!

%% Regole

`IF`	*azione-if*
`WHILE`	*azione-while*
...	
`IDE`	*azione-ide*

Se lo scanner trova il match `WHILE`, riconosce la parola riservata perché è listata prima; se trova `WHILES`, prende il match più lungo (IDE, in questo caso)!

Esempio di file `lang.l` meno banale

```
cifra       [0-9]
cifre       [0-9]+
ide         [a-zA-Z][a-zA-Z0-9]*
separatore  [ \t\n]*
razionale   {cifre}"."{cifre}
bin         [+*]
%%
{cifre}                         { printf("<NUM,%s>", yytext); }
{razionale}                     { printf("<FRACT,%s>", yytext); }
"+"|"-"|"*"|"/"|"while"|";"|
"("|")"|"{"|"}"|":="            { printf("<%s>", yytext); }
{ide}                           { printf("<IDE,%s>", yytext); }
{separatore}                    { printf(" "); }
{bin}|"-"                       { printf("<OP2,%s>", yytext); }
```

Semplici dichiarazioni, poche regole, nessuna funzione ausiliaria.
Da notare che `yytext` è la variabile che punta al primo carattere del lessema, mentre la lunghezza del lessema è indicata dalla variabile `yyleng`.

Esempio di funzionamento dello scanner prodotto

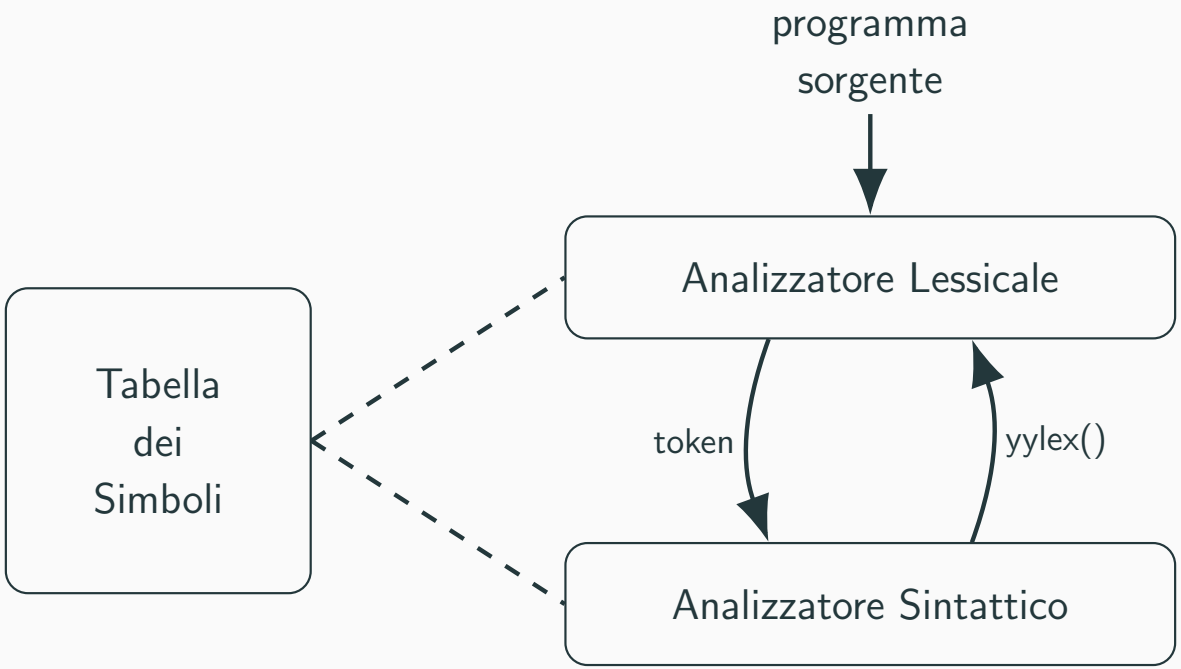
Applicando lo scanner prodotto da Lex, a partire dal file `lang.l` della pagina precedente, sul seguente frammento di programma

```
pigreco := 3.14;
temp1 := pigreco * (temp1 + 2);
```

si genera la sequenza di token

```
<IDE, pigreco> <:=> <FRACT, 3.14> <;>
<IDE, temp1> <:=> <IDE, pigreco> <OP2, *>
<(> <IDE, temp1> <OP2, +> <NUM, 2> <)> <;>
```

Utilizzo con YACC



- Il programma generato da Lex non è usato da solo.
- È una subroutine per il parser generato da YACC (generatore di parser).
- Alcune variabili comuni ai due programmi (e.g. `yylval`) permettono ai due programmi di scambiarsi informazioni.
- `lex.yy.c` è usato dall'analizzatore sintattico generato da YACC on-demand, richiedendo il token successivo alla bisogna.
- `yylex()` è la funzione che permette di invocare `lex.yy.c` e restituisce il nome del token, mentre il valore del token è condiviso nella variabile `yylval`

Esempio di uso con YACC

```
%{
#define NUM 1
#define FRACT 2
#define IDE 3
#define OP2 4
#define PIU 41
#define PER 42
#define MEN 43
%}
cifra           [0-9]
cifre           [0-9]+
ide             [a-zA-Z][a-zA-Z0-9]*
separatore      [ \t\n]*
razionale       {cifre}"."({cifre})
%%
{separatore}    {  }
{cifre}         { yylval = inConstTable(); return(NUM); }
{razionale}     { yylval = inConstTable(); return(FRACT); }
{ide}           { yylval = inSymTable(); return(IDE); }
+               { yylval = PIU; return(OP2); }
*               { yylval = PER; return(OP2); }
-               { yylval = MEN; return(OP2); }
%%
int inConstTable() {
    /* Converte la stringa puntata da yytext in
       uno (NUM) o due (FRACT) interi, li inserisce
       nella tabella delle costanti e restituisce
       (come int) un puntatore all'elemento inserito */ }
int inSymTable() {
    /* Inserisce nella tabella dei simboli la
       stringa puntata da yytext e restituisce
       (come int) un puntatore all'elemento inserito */ }
```

} *dichiarazioni ausiliarie*

} *definizioni regolari*

} *regole*

`lex.yy.c` mette nella variabile `yylval` il puntatore alla tabella dei simboli (o delle costanti) dove ha messo il lessema appena individuato

} *funzioni ausiliarie*

Proprietà algoritmiche dei linguaggi regolari

Proprietà algoritmiche dei linguaggi regolari

Finora non abbiamo dimostrato che esistono linguaggi **non regolari**.

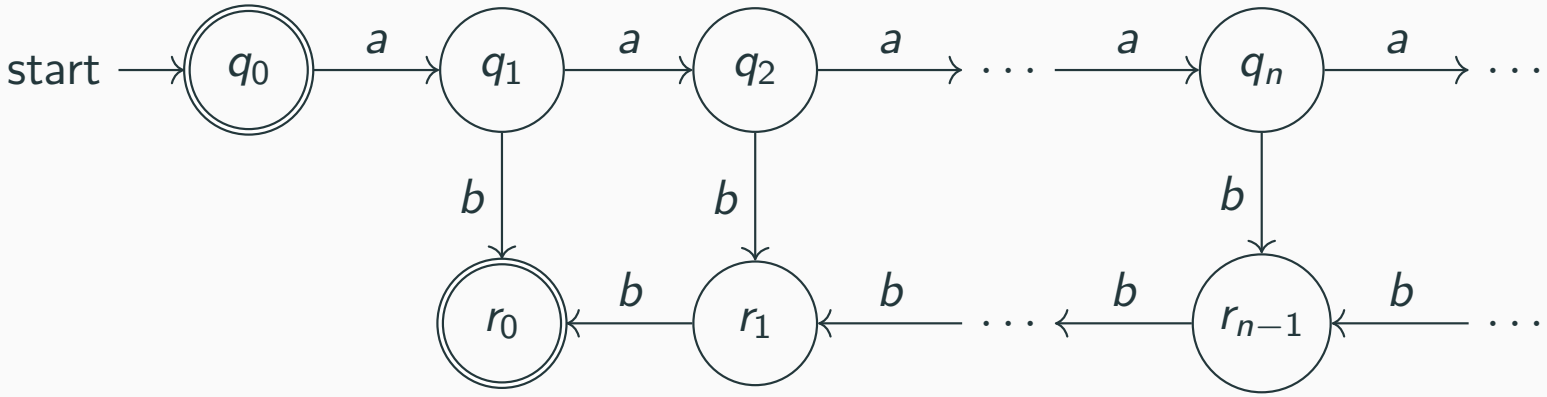
- Sappiamo che una grammatica regolare è un caso speciale di grammatica libera, quindi sicuramente
  linguaggi regolari $\subset$ **linguaggi liberi**
- Ora vedremo una proprietà algoritmica dei linguaggi regolari che sarà utile per dimostrare che certi linguaggi, non possedendola, non sono regolari!
- Ad esempio, $L = \{a^n b^n | n \geq 0\}$ è sicuramente libero perché generato dalla grammatica libera

  $$S \to \epsilon \mid aSb,$$

  ma dimostreremo che **non è regolare**, ovvero che nessun NFA/DFA può riconoscerlo.

Esempio

- Intuitivamente, $L = \{a^n b^n | n \geq 0\}$ non può essere regolare perché c'è necessità di "contare" illimitatatmente (per ogni $n \geq 0$, devo verificare che le a e le b siano in numero uguale) e per farlo servirebbero **infiniti** stati!



- Un NFA/DFA ha solo un numero finito di stati, per cui può "contare" solo tanto quanto gli stati che ha.
- **Attenzione**: il linguaggio $L = \{a^n b^m \mid n, m \geq 0\}$ è regolare e corrisponde alla espresione regolare $a^* b^*$.

Proprietà algoritmica di tutti i linguaggi regolari

Pumping Lemma

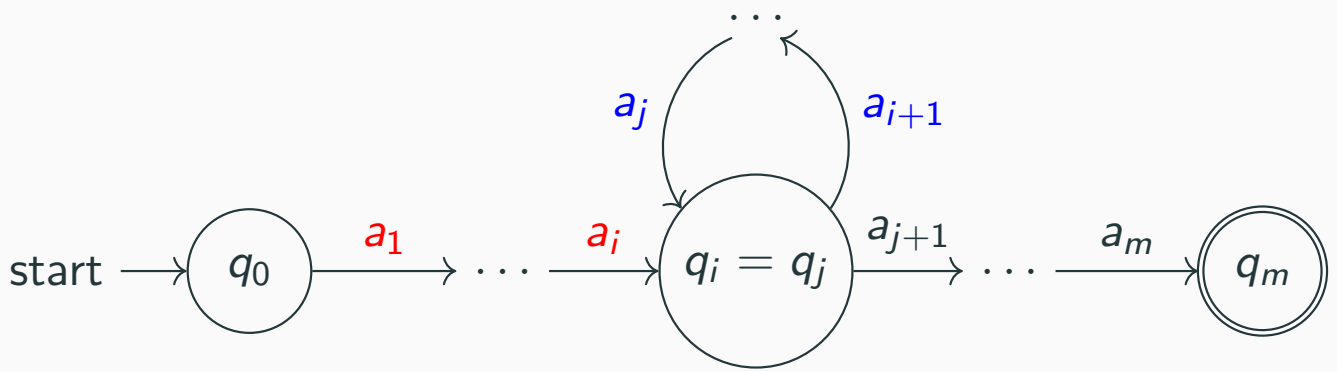
Se L è un linguaggio regolare, allora $\exists N > 0$ tale che $\forall z \in L$ con $|z| \geq N$, $\exists u, v, w$ tali che:

- $z = uvw$
- $|uv| \leq N$
- $|v| \geq 1$
- $\forall k \geq 0 : uv^k w \in L$.

Inoltre, N è minore o uguale al numero di stati del DFA minimo che accetta L.

Dimostrazione: Sia $N = |Q_M|$ dove M è il DFA minimo che accetta L e sia $z = a_1 a_2 ... a_m \in L$ con $m \geq N$.
Quindi $q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} q_2 \xrightarrow{a_3} ... \xrightarrow{a_m} q_m \in F$.
Ora, questo cammino tocca $m + 1$ stati (con indici che vanno da 0 a m), e quindi $m + 1 > N$.
Allora devono esistere i, j con $i < j$ tali che $q_i = q_j$.

Dimostrazione



- Individuiamo $u = a_1 \dots a_i$, $v = a_{i+1} \dots a_j$, e $w = a_{j+1} \dots a_m$. Ovviamente $z = uvw$.
- Poiché $i < j$, allora il ciclo $v = a_{i+1} \dots a_j$ è tale che $|v| \geq 1$.
- La condizione $|uv| \leq N$ mi dice che sicuramente un ciclo deve essersi formato entro N passi (visto che se il cammino che riconosce uv è di lunghezza N, allora tocca $N + 1$ stati).
- Infine la condizione $\forall k \geq 0 : uv^k w \in L$ si vede visivamente dal diagramma sopra:

$uv^0 w = uw \in L$	non percorro il ciclo
$uv^1 w = uvw = z \in L$	percorro il ciclo una volta
...	
$uv^k w \in L$	percorro il ciclo k volte

q.e.d.

Osservazioni

Osserviamo che

- Se L è finito, allora non esiste nessuna $z \in L$ con $|z| \geq N$ e quindi "l'implicazione" del pumping lemma è vera perché la premessa è falsa.
- Se $\exists z \in L$ con $|z| \geq N$, allora M riconosce un linguaggio infinito, grazie all'ultima proprietà espressa dal pumping lemma.

Come usare il pumping lemma per dimostrare che un linguaggio non è regolare?
Il pumping lemma dice che se L regolare, allora vale P, dove P è la proprietà espressa dal lemma.
L'implicazione che useremo è quindi se P non vale, allora L non è regolare!

Negazione del Pumping Lemma

Se $\forall N > 0, \exists z \in L$ con $|z| \geq N$, tale che

- $\forall uvw$, se
  - $z = uvw$
  - $|uv| \leq N$
  - $|v| \geq 1$
- allora $\exists k \geq 0 : uv^k w \notin L$,

allora L non è regolare.

Esempio: $\{a^n b^n | n \geq 1\}$ non è regolare

- Fissiamo N generico ($\forall N > 0$)
- Scegliamo $z = a^N b^N$ ($\exists z \in L$ con $|z| \geq N$)
- Guardiamo tutte le possibili scomposizioni di z in tre sottostringhe ($\forall uvw$) tali che
  - $z = uvw$
  - $|uv| \leq N$
  - $|v| \geq 1$.
- La condizione $|uv| \leq N$ impone che u e v siano fatte di sole "a", quindi $v = a^j$ con $j \geq 1$.
- Quindi, $\exists k = 2$ tale che $uv^2w = uvvw = a^{N+j} b^N \notin L$.

Ne segue che L non è regolare!

Esempio: $L_1 = \{a^{n^2} | n \geq 1\}$ non è regolare

- Fissiamo N generico ($\forall N > 0$)
- Scegliamo $z = a^{N^2}$ ($\exists z \in L$ con $|z| \geq N$)
- Per ogni possibile scomposizione di $z = uvw$ tale che
  - $|uv| \leq N$
  - $|v| \geq 1$

  deve esere che $1 \leq |v| \leq N$.
- Prendiamo $k = 2$, cioè consideriamo uv^2w. Vale che

  $$|z| = N^2 < |uv^2w| = |z| + |v| \leq N^2 + N < N^2 + 2N + 1 = (N+1)^2$$

  cioè uv^2w ha una lunghezza strettamente compresa tra N^2 e $(N+1)^2$. Nessun quadrato perfetto esiste tra N^2 e $(N+1)^2$!
  Quindi $uv^2w \notin L_1$.

Ne segue che L_1 non è regolare!

Esempio: $L_2 = \{a^{2^n} | n \geq 1\}$ non è regolare

- Fissiamo N generico ($\forall N > 0$)
- Scegliamo $z = a^{2^N}$ ($\exists z \in L$ con $|z| \geq N$)
- Per ogni possibile scomposizione di $z = uvw$ tale che
  - $|uv| \leq N$
  - $|v| \geq 1$

  deve esere che $1 \leq |v| \leq N$.
- Prendiamo $k = 2$, cioè consideriamo uv^2w. Vale che

$$|z| = 2^N < |uv^2w| = |z| + |v| \leq 2^N + N < 2^N + 2^N = 2^{N+1}$$

  cioè uv^2w ha una lunghezza strettamente compresa tra 2^N e 2^{N+1}.
  Nessun numero compreso fra 2^N e 2^{N+1} è una potenza di 2.
  Quindi $uv^2w \notin L_2$.

Ne segue che L_2 non è regolare!

Esempio: $L_3 = \{ww | w \in \{a, b\}^*\}$ non è regolare

- Fissiamo N generico ($\forall N > 0$)
- Scegliamo $z = a^N b^N a^N b^N$ ($\exists z \in L$ con $|z| \geq N$)
- Per ogni uvw tali che:
  - $z = uvw$
  - $|uv| \leq N$ e
  - $|v| \geq 1$

    deve essere che v è fatta solo di "a" con $1 \leq |v| \leq N$.

    Sia $v = a^j$ con $j \geq 1$.
- Prendiamo $k = 2$

$$uv^2w = a^{N+j} b^N a^N b^N \notin L_3.$$

Ne segue che L_3 non è regolare!

Esempio: $L_4 = \{ww^R | w \in \{a, b\}^*\}$ non è regolare

- Fissiamo N generico ($\forall N > 0$)
- Scegliamo $z = a^N b^N b^N a^N$ ($\exists z \in L$ con $|z| \geq N$)
- Per ogni uvw tali che
  - $z = uvw$
  - $|uv| \leq N$ e
  - $|v| \geq 1$

    deve essere che v è fatta solo di "a" con $1 \leq |v| \leq N$.

    Sia $v = a^j$ con $j \geq 1$.
- Prendiamo $k = 2$

$$uv^2w = a^{N+j} b^N b^N a^N \notin L_4.$$

Quindi L_4 non è regolare!

Esempio: $L_5 = \{a^n | n$ è un numero primo$\}$ è regolare?

- Fissiamo N generico ($\forall N > 0$)
- Scegliamo $z = a^p$ con p primo e $p \geq N + 2$ ($\exists z \in L$ con $|z| \geq N$)
- Per u, v, w tali che $z = uvw$ e
  - $|uv| \leq N$
  - $|v| \geq 1$, deve esere che $1 \leq |v| \leq N$. Sia $m = |v|$. Allora $|uv^0w| = |uw| = p - m$, che deve essere un numero primo se $uv^0w \in L_5$.
- Consideriamo $k = p - m$. Vale che

$$\begin{aligned} |uv^{p-m}w| &= |uw| + (p-m) \cdot |v| = \\ &= (p-m) + (p-m) \cdot m = \\ &= (p-m) \cdot (1+m) \end{aligned}$$

  $(p-m) \cdot (1+m)$ non è primo se $p - m \neq 1$ e $1 + m \neq 1$. Visto che $m \geq 1$, allora $(1 + m) > 1$ e dato che $p \geq N + 2$ e $m \leq N$, allora $(p - m) \geq 2$, quindi davvero $(p - m) \cdot (1 + m)$ non è primo.
  Quindi $uv^{p-m}w \notin L_5$.

Ne segue che L_5 non è regolare!

Altre proprietà dei Linguaggi Regolari

Teorema
La classe dei linguaggi regolari è chiusa per:

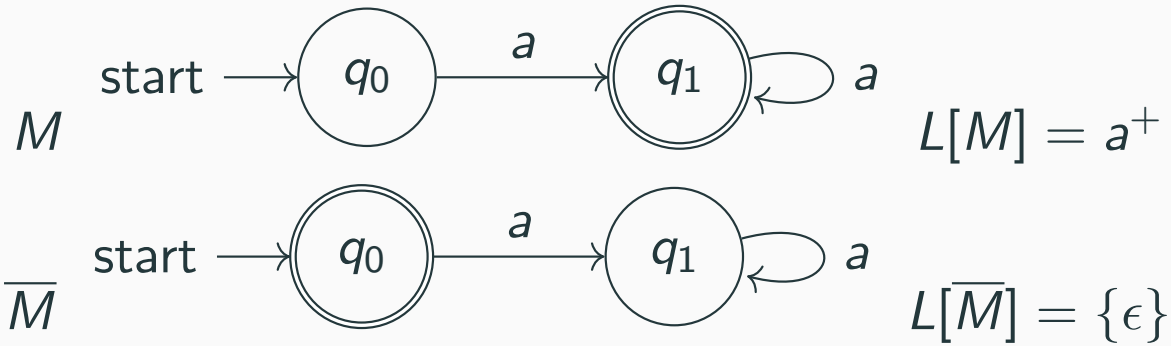
1. unione,
2. concatenazione,
3. stella di Kleene (o ripetizione),
4. complementazione,
5. intersezione.

Dimostrazione: (1), (2), (3) sono ovvie, perché discendono direttamente dal fatto che questi tre sono operatori delle espressioni regolari. Ad esempio, se L_1 e L_2 sono regolari, allora esistono s_1 e s_2 espressioni regolari tali che $L_1 = \mathcal{L}[s_1]$ e $L_2 = \mathcal{L}[s_2]$. Ma allora:

1. $L_1 \cup L_2 = \mathcal{L}[s_1|s_2]$, cioè $L_1 \cup L_2$ è regolare.
2. Similmente, $L_1 \cdot L_2 = \mathcal{L}[s_1 \cdot s_2]$, cioè $L_1 \cdot L_2$ è regolare.
3. E anche $L_1^* = \mathcal{L}[s_1^*]$, cioè L_1^* è regolare.

Dimostrazione – continua

- (4) è vera perchè, dato un DFA $M = (\Sigma, Q, \delta, q_0, F)$ tale che $L = L[M]$, possiamo costruire il DFA $\overline{M} = (\Sigma, Q, \delta, q_0, Q \setminus F)$ tale che $\overline{L} = L[\overline{M}]$. Infatti $w \in L[M]$ se e solo se $w \notin L[\overline{M}]$ e quindi $L[\overline{M}] = \Sigma^* \setminus L[M]$. Esempio:



- (5) discende dalla legge di De Morgan

$$L(M_1) \cap L(M_2) = \overline{\overline{L(M_1)} \cup \overline{L(M_2)}}$$

Poiché abbiamo già dimostrato che i linguaggi regolari sono chiusi per unione e complementazione, ne segue che sono chiusi anche per intersezione.

Dimostrazione alternativa della chiusura per $\cap$

Dati i seguenti due DFA

- $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$ ed
- $M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$
  costruiamo il DFA prodotto
- $M = (Q_1 \times Q_2, \Sigma, \delta, (q_{01}, q_{02}), F_1 \times F_2)$,
  - dove $\delta((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a))$.
- Si può dimostrare che $L[M] = L[M_1] \cap L[M_2]$.

Come sfruttare la chiusura per $\cap$

La chiusura per intersezione può essere utile per dimostrare che un linguaggio non è regolare.
Infatti, se $L \cap L_{reg} = L_{non-reg} \Rightarrow L$ non è regolare.
Esempio:
$L = \{w \in \{a, b\}^* |$ in w occorrono tante "a" quante "b"$\}$ è regolare?
Se lo fosse, allora anche $L \cap a^* b^*$ dovrebbe essere regolare. Ma $L \cap a^* b^* = \{a^n b^n | n \geq 0\}$, che abbiamo appena dimostrato che non è regolare.
Quindi L non è regolare!

Altre proprietà dei linguaggi regolari

Per i linguaggi regolari si possono decidere (verificare algoritmicamente) le seguenti proprietà (indichiamo M, M_1, M_2 dei DFA):

- $w \in L[M]$? Basta leggere w e vedere se si è raggiunto uno stato finale.
- $L[M] = \emptyset$? Basta verificare se esiste un cammino aciclico dallo stato iniziale ad uno stato finale.
- $L[M] = A^*$? vale se e solo se $L[\overline{M}] = \emptyset$.
- $L[M_1] \subseteq L[M_2]$? vale se e solo se $L[\overline{M_2}] \cap L[M_1] = \emptyset$.
- $L[M_1] = L[M_2]$? vale se e solo se $L[M_1] \subseteq L[M_2]$ e $L[M_2] \subseteq L[M_1]$ (o anche: costruisco i due DFA minimi e vedo se sono isomorfi)
- $L[M]$ è infinito? Basta verificare se esiste una parola $z \in L[M]$ tale che $n \leq |z| \leq 2n$ dove $n = |Q|$.